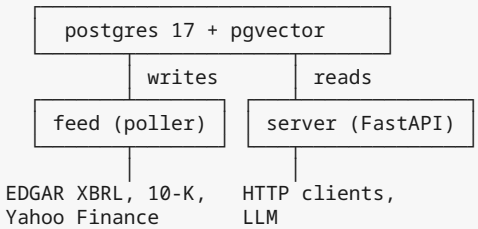
Design writeup

Architecture

The service is a Postgres-backed one that ingests three public sources: SEC XBRL facts, SEC 10-K narratives, and Yahoo Finance prices. It answers questions that span the reported numbers and the discussion in the filings. It is useful for an analyst who wants to ask questions in natural language and receive an answer that is honest about which numbers come from data and which do not.

There are two Python services sharing one database: a long-running poller that refreshes the sources, and a read-only FastAPI service with a natural-language endpoint on top.



It is split this way because ingestion is pull-based and slow-moving, serving is read-only and must stay up. The feed never serves, and the server never fetches from a public source. The only thing they share is the schema.

Runtime choices

Sync everywhere. edgartools and psycopg are both blocking, so the feed runs single-threaded and the API handlers are plain `def`. FastAPI runs those in a thread pool, which is enough for a handful of analysts: 40 reads at 40-way concurrency finish in 49 ms.

Data model

Table	Holds	Why this schema
`entities`	CIK, name, ticker	Ticker is display-only and not unique; GOOGL and GOOG share a CIK
`concepts`	taxonomy + tag name	Reference data, unique on `(taxonomy, name)`
`facts`	one row per fact per filing	Natural key `(cik, concept, unit, start, end, accession)` with `NULLS NOT DISTINCT`
`filing_sections`	Item 1A / Item 7 text	One row per filing and section, whole text
`filing_chunks`	retrieval windows	Generated `tsvector`, nullable `vector(1536)`
`prices`	daily bars	Mutable: rewritten each cycle

Two further decisions shaped the schema:

Narrative is stored twice: whole and chunked. Whole sections answer "what new risk factors did they add", which is a comparison of two contiguous texts; chunks answer "which paragraphs discuss tariffs", which needs ranking. Two risk-factor sections are ~115,000 characters each, so the comparison can never happen in a prompt.

Prices are rewritten, facts are appended. Yahoo back-adjusts its entire history when a stock split happens, so a stored bar is not immutable. `prices` updates/inserts with `DO UPDATE`; `facts` uses `DO NOTHING`.

Three data sources

- **XBRL.** One request per company to `data.sec.gov`, ~25,000 facts each, normalized into entity, concepts, and facts inside a single transaction per ticker.
- **10-K narrative.** The two most recent unamended 10-Ks per company; Item 1A and Item 7 extracted, whitespace collapsed, then chunked into ~4,000-character windows with 10% overlap.
- **Prices.** Ten years of daily bars per ticker in one call, ~12,500 rows total, refetched and rescaled every cycle. Yahoo through yfinance because it needs no API key and returns adjusted closes. Alpaca or Alpha Vantage would put a credential between a fresh clone and a working stack, for the same daily data.

A cycle takes about fifteen seconds for all five companies and makes a few dozen requests, well inside SEC's fair-access limits. Re-running is idempotent: a second cycle reports `0 new rows`.

The feed polls every hour, far more often than any source changes. It means a new filing appears within the hour, but most cycles re-parse both 10-Ks and re-send ~126,000 fact rows to find nothing new. A "newest accession unchanged" check would remove that; I left it out because the whole cycle costs seconds.

The cross-source join

The valuation endpoint joins a Yahoo price with EDGAR fundamentals:

```
market_cap  = latest close × shares outstanding
trailing_pe = latest close ÷ latest annual diluted EPS
trailing_ps = market_cap ÷ annual revenue
```

The share count needs two tags to stay fresh:

`dei:EntityCommonStockSharesOutstanding` comes from the cover page and refreshes quarterly, but Alphabet reports shares per class with a dimension, and the pipeline stores consolidated facts only, so GOOGL has no cover-page count at all. `us-gaap:CommonStockSharesOutstanding` covers every company, but NVDA tags it only annually, so its newest value is a year old while the price is current. I take the freshest period available.

Every valuation response returns its inputs and their dates (`price_date`, `period_end`, `shares_as_of`) rather than a bare ratio.

Where the LLM is, and isn't

`POST /ask` runs a five-step tool loop over five tools: fundamentals, a computed ranking, valuation, filing search, and a year-over-year section diff.

Step	LLM	Why
Out-of-scope detection	No	A keyword guard refuses guidance and forecasts before any model call. Models will invent a plausible number rather than admit they cannot know one.
Tool selection	**Yes**	Deciding whether a question needs numbers, text, or both
Arithmetic, ranking, diffing	No	Computed in SQL or Python and returned as results
Final prose and citations	**Yes**	Language is what it is good at
Number verification	No	A check, not a judgment: every figure in the answer must appear in a tool result

The model never computes, so the only numbers it can report are ones a tool produced. The response carries `tools_used`, `citations` (ticker, section, period, accession), and a `grounding` block. `grounding.ok` is advisory. It flags figures it cannot trace back to a tool result.

Tradeoffs and what was left out

- **Keyword list guard instead of classifier for rejection.** Given more time I would expand the prompt rejection handling to use a classifier model instead of regex.
- **Sync handlers, not async.** An async refactor (async driver, async HTTP client, async tools) would raise throughput under concurrency, but not make any single answer faster. With a handful of users and their requests, the 40-thread pool is far above the load.
- **Annual EPS, not a TTM rollup.** P/E uses the latest reported annual EPS, so it can be up to a year old; `period_end` is returned beside it so the age is visible.
- **No vector embeddings.** Full-text search answers these questions, needs no API key, and works when the endpoint in front of the model exposes no `/embeddings` route. The column and index exist; nothing depends on them.
- **No MCP server.** The service runs its own agent loop and is the only consumer of the five tools, so an MCP server would add a second transport with no MCP client to serve. The tool schemas are transport-neutral, so wrapping them for MCP clients is an additive change later.
- **Additional data sources: no quarterly metrics, 8-K, insider transactions.** I would add these given more time.